

IMMC 2022 International Round

Team Control Number: 22272397

“Efficiency is time and time is money” -- IMMC

I. Summary

Aircrafts are some of the most efficient ways to travel on Earth. These huge machines can transport numerous passengers from one place to another in mere hours, and are efficient in the transporting, yet the boarding and disembarking processes take up valuable time, both for the passengers, and also for the aircraft crew. As IMMC said, “in air transportation, efficiency is time and time is money.” Even the smallest delays caused by a small jostle or trip can accumulate into severe time lost if no action is taken to improve the methods. Therefore, our team is given the task to explore, experiment, and establish the optimal boarding and disembarking methods on various types of aircraft. Through extensive research and modeling, we found out that such delays can be greatly shortened through the variation of methods.

In this paper, we will mainly focus on the standard “Narrow Body” passenger aircraft, where there is one aisle with three seats on each side. We will also be studying two other kinds of aircraft: the “Flying Wing” passenger aircraft, and the “Two Entrance, Two Aisle” Passenger Aircraft, and find the optimal boarding and disembarking method for each. In addition, we will be considering what will happen with the factor caused by the pandemic, where passengers will sharply decrease to only 70%, 50%, 30%. Finally, we will write a letter to the Airline Executive in order to report our findings and give them advice and recommendations in boarding and disembarking aircrafts.

Creating the boarding and disembarking method will need the time each passenger takes to sit in their respective seats. Then, the passenger who needs the longest time to sit in would be identified. Calculating the time each passenger needs to board and disembark an aircraft involves calculating both the speed and the time waiting when people stop due to clogging of the waiting line. During our research and discussions, we found that the time of boarding and disembarking an aircraft depends on two main factors. Luggage placing, where passengers must stop to wait for one person to put in their luggage, and seat shuffle, where the passengers must step back outside to the aisle from their seats in order to allow passengers sitting on an inner seat to get in.

Our goal is to find the most optimal way to board and disembark the aircraft. Time is important, but it is not the only consideration needed to be taken into account, since a priority for our team is to make the simulation and experiments as realistic as possible. Some methods are not practical in actual use, for example, the Steffen Perfect method. We will also consider factors like the need for family members to be as close to each other as possible, the number of groups in which the passengers would be divided into, and the overall rowdiness of the aircraft.

The optimal way will be considered through all of the factors above.

Contents

I. SUMMARY	1
III. DEFINING THE PROBLEM.....	3
3.1 QUESTION RESTATEMENT	3
3.2 ASSUMPTIONS.....	4
3.3 VARIABLES	5
IV. MODELING BOARDING AND DISEMBARKING AIRCRAFT.....	5
V. NARROW BODY PASSENGER AIRCRAFT	7
5.1 TIME NEEDED FOR EACH BOARDING METHODS.....	7
5.2 SENSITIVITY ANALYSIS	8
5.3 SITUATION OF MORE LUGGAGE.....	9
5.4 OTHER BOARDING METHODS	9
5.5 OPTIMAL DISEMBARKING METHOD.....	10
VI. OTHER PASSENGER AIRCRAFT.....	11
6.1 FLYING WING PASSENGER AIRCRAFT	11
6.2 TWO ENTRANCE TWO AISLE PASSENGER AIRCRAFT	11
VII. PANDEMIC SITUATIONS.....	12
VIII. LETTER.....	13
IX. STRENGTHS.....	14
X. WEAKNESSES	14
XI. APPENDIX.....	15
XII. WORKS CITED	28

II. Introduction

Efficiency, the state of working in a well-organized and competent way [1], is undoubtedly an important aspect during the boarding and disembarking of an aircraft. Due to various factors like accidents, delays, seating positions, and row numbers, the time needed in total for a plane to be completely seated would become increasingly long, which is an unwanted thing for those in a hurry. Therefore, in order to decrease and minus the possible time which can increase overall time needed, our team is to create a model which can find out the most efficient way in both boarding and disembarking a plane.

III. Defining the Problem

3.1 Question restatement

Our team is to complete five tasks:

1. Calculate the total boarding and disembarking time and create a model which outlines the different aspects which can affect the time taken.
2. Apply the model to the standard “narrow body” aircraft and, first, compare the boarding times for the boarding methods of random boarding, boarding by section, and boarding by seat. Second, we would need to analyze the how the times vary by differing numbers. Third, through the analysis and two additional boarding methods, explain the optimal boarding method and justify.
3. Modify the model accordingly to the “Flying Wing” passenger aircraft and the “Two-entrance, Two-aisle” aircraft and recommend the most favorable boarding method for each kind of aircraft.
4. Observe whether or not the boarding method would change if the number of passengers were limited to a specific percent.
5. Write a one-page recommendation letter to an airline executive describing our results, recommendations, and rationale about passenger aircraft boarding and disembarking in a non- mathematical way.

3.2 Assumptions

In designing our model, we decided on several basic assumptions. Listed below are the assumptions and our justifications for them.

Assumption	Justification
Each passenger would take five seconds to lift one piece of luggage onto the overhead cabins.	We make this assumption because it is impossible to measure and predict the lifting strength of every passenger. In addition, by doing so, the function concerning the amount of time needed per passenger to stow away the luggage would be made easier due to the decrease in complexity which can make the overall process more understandable and efficient.
The passengers' walking speed is only affected by their age.	Due to the impossibility to measure the physical conditions of each passenger and how their walking speed is affected by those different factors, this assumption eliminates the things which cannot be calculated and leaves the things able to be searched in order to ensure that the numbers used would be accurate.
Children do not have any luggage	Since children have an average height of about 1.21m [2], it would be near impossible for them to lift their bags onto the overhead bins, therefore it would take up an unpredictable amount of time that is both unnecessary and unwanted in the model in case of messing up the calculation.
An infant is to be carried by an adult the entire time	FAA rules, and the airlines, do not require that infants and toddlers under the age of 2 be in a separate seat, [3], and therefore because of this, the infants do not take up one seat on the airplane, therefore they are assumed to be carried by the adult throughout the entirety of the flying session.
As long as the passenger reaches his or hers aligning row, he or she would go in without waiting people at the front to clear enough space for the person already sitting in the row to get out in.	First, it would be challenging to calculate the number of people at the back for each row in the aircraft, and second, to be realistic, it is possible to squeeze past the passenger, therefore enabling the exchange of position to get to the passengers' seats accordingly.

3.3 Variables

T_{distance}	Time needed to go a certain row
T_{luggage}	Time needed for the passenger to lift one piece of luggage onto the overhead bins
T_{swap}	The time it takes for seat shuffle
T_{delay}	Time delayed due to passenger of smaller-numbered row stowing away luggage
$T(p)$	Function for the total time per passenger to walk towards corresponding row, stow away luggage, and sit on the seat
pas_{row}	The row of passenger's seat
pas_{col}	The column of passenger's seat
pas_{sec}	The section of the passenger's seat
pas_{spd}	The speed of the passenger
$\text{enter}_{\text{row}}$	The row where passengers are entering
exit_{row}	The row where passengers are exiting

IV. Modeling Boarding and Disembarking Aircraft

The programming part of the model is a simulation of the actual scene when boarding and disembarking the plane.

INPUT

The dimensions of the plane, the number of passengers who have carry-on bags, their ages, and the onboarding and disembarking method are the inputs (or variables) of the program, outputting the boarding and disembarking time. To calculate the time of boarding is equivalent to the time it takes for the last person sits on his/her seat while the time of disembarking is the same as the time it takes for the last person to leave the plane.

PROCESS

First, the program will process the inputs. For instance, passengers' ages are converted into speed (m/s) through fitting. The dimensions of the plane are utilized to create a list of possible seat numbers.

SIMULATION

Next, the program will arrange the possible seat numbers to suggest a specific order of how the passengers will onboard. This part varies due to the differences in various boarding methods. All passengers are stored in one list (or pList in the program) with its corresponding time list (tList), recording the time it takes for the passenger to sit on his/her seat. It includes the time for the passenger to walk to his/her seat (T_{distance}), the time wasted on waiting in line (T_{delay}), and the time spent on seat shuffling (T_{swap}). The general function to calculate the onboarding time is $T(p)$.

$$T(p) = T_{\text{distance}} + T_{\text{delay}} + T_{\text{luggage}} + T_{\text{swap}}$$

1. T_{distance}

For entering,

$$T_{\text{distance}} = |\text{pasrow} - \text{enterrow}| \times 0.8128 / \text{passpd}$$

For exiting,

$$T_{\text{distance}} = |\text{pasrow} - \text{enterrow}| \times 0.8128 / \text{passpd}$$

This formula is adapted from " $t = \text{avg}(d)/\text{avg}(s)$ " 0.8128 is a constant, converting the difference between the numbers of rows into meters[3]. Passengers' speeds are extrapolated by fitting, using a experimental data from a paper from Hartford Hospital [4].

2. T_{delay}

T_{delay} is caused when the passenger in front is putting his/her carry-on bags into overhead bins, thus blocking the aisle. T_{delay} is equal for every passenger waiting behind and adds to their time total of onboarding.

3. T_{luggage}

T_{luggage} refers to the time spent to put carry-on bags into overhead bins. In this case, we assume that every passenger spent an average of 5 seconds to situate their bags (if they have bags as prerequisite) because every passenger has different weights of bags and different strengths.

4. T_{swap}

T_{swap} is the time exhausted in seat shufflings, which occurs when a person sitting on the same row is blocking your way to your seat. This person will have to leave his/her seat and make way for you to sit on your seat, thus resulting in some time being wasted. Since it happens very commonly and frequently when onboarding a plane, we decide to consider T_{swap} into the total onboarding time, but because there are no related studies about seat shuffling, we simulated the exact scene of seat shuffling and got an average result of 6.192 seconds per swap.

In the experiment, two people participated. The first person sat on the window seat, and stood up, walked out the aisle, letting the second person sit into the seat, then sat back down on his seat. We initiated this experiment three times and calculated the average.

OUTPUT

After simulating the onboard and disembark of the same plane for 100 times, the averages of onboarding time and disembarking time are computed and finally outputted. Other data like the 5th and 95th percentile can also be easily acquired as well.

MODIFY

The only variable that dramatically changes the program is the dimension of the plane, for it affects the possible seats for passengers to sit, T_{distance} , and disembarking time (the exits are in different places). Therefore, there are three different programs specialized for "Narrow body," "Flying Wing," and "Two-Entrance, Two Aisle" aircrafts. Other minor

adjustments, such as the number of passengers bringing carry-on bags, the passengers' ages, and the onboarding methods, are modified in those programs.

V. Narrow Body Passenger Aircraft

5.1 Time Needed for each Boarding Methods

The question requires us to apply our model to the following methods:

1. Random (unstructured) boarding
2. Boarding by Section: Examine varying the order of aft section (rows 23-33), middle section (rows 12-22), and bow section (rows 1-11).
3. Boarding by Seat: In the order of window seats (A and F), middle seats (B and E), and aisle seats (C and D).

The factors that save time can be categorized into seat shuffle and luggage placing. In the following methods, only Boarding by Seat actually decreases some factors that affect time.

The first boarding method can be done by randomly generating the coordinates of the

Method	Board by Random	5th percentile	95th percentile
Scenario	1	827	1059
	2	831	1067
	3	818	1070
	4	816	1071
	5	852	1098
Average		828.8	1073
List Average	955		

seat to each passenger. Running our code, rounding the numbers to a whole number, the results are shown on the table to the left.

The second boarding method can be done by randomly generating the order which the passenger is going to the given rows of the seat. By this, each of the sections have an order in which the people in row 23-33 goes the first, the people in row 12-22 goes the second, the people in row 1-11 goes the last. The results are shown on the table to the right.

Method	Board By Section	5th percentile	95th percentile
Scenario	1	929	1190
	2	930	1124
	3	901	1124
	4	919	1123
	5	936	1121
Average		923	1136.4
List Average	1030		

The third boarding method can be done by randomly generating the order which the

Method	Board by Seat	5th percentile	95th percentile
Scenario	1	394	514
	2	396	492
	3	404	510
	4	386	518
	5	400	495
Average		396	505.8
List Average	452		

passenger is going to the given columns of the seat (ABCDEF). By this each sections have an order of first passengers on the window seats A and F, then passengers on the middle seat B and E, at last passengers on the aisle seat C and D. The results are shown on the table to the left.

Comparing all of the data, we found out that the Boarding by Seat method is the fastest. In all, the results are the following:

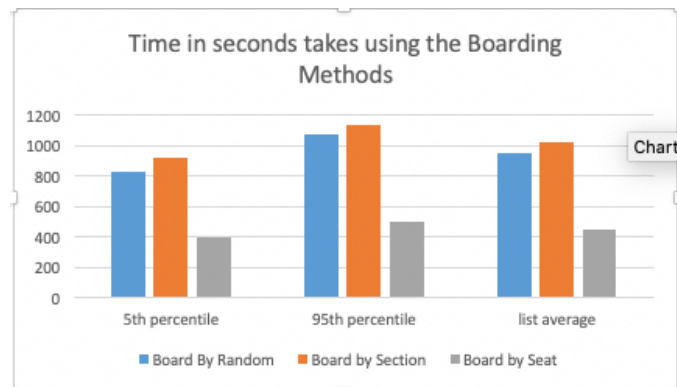
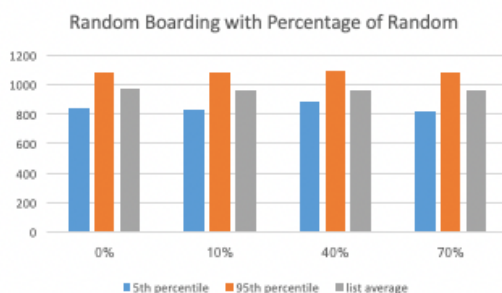


Figure 3: Bar graph comparing the time (seconds) between the methods of boarding given

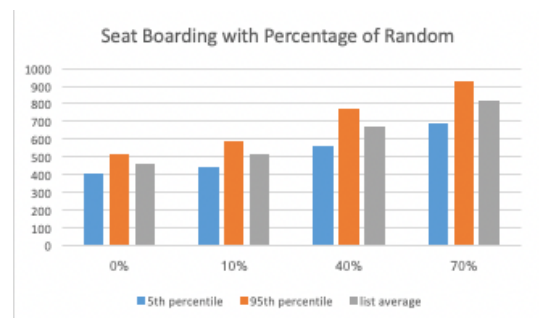
5.2 Sensitivity Analysis

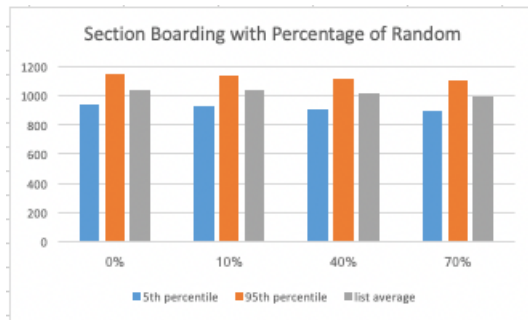
It is common for people not following directions. When boarding a plane, chaos and disorder can all happen. Using the model, we have also recorded each method's time needed when people who don't follow directions percentage is at 10%, 40%, and 70%. The results are the following:



Boarding by seat have an increase of time needed, as the percentage of people who didn't follow the rules increases. This means, that at a random rate, more seat shuffle appears.

Boarding by random with the percentage of random didn't change. It still levels at around the same place. It is easy to understand the results, as randomizing an already random sequence won't change anything.



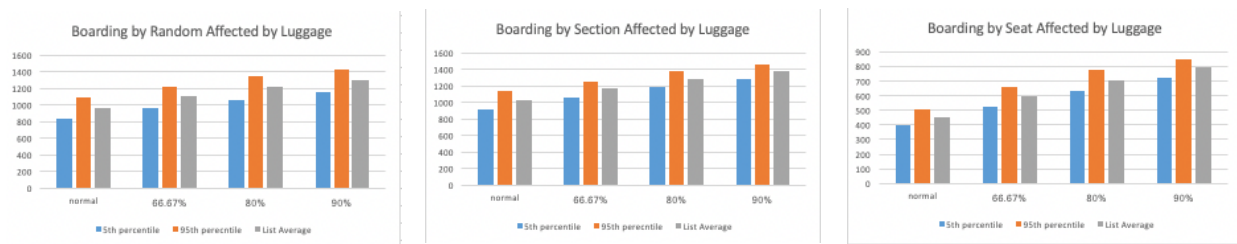


Boarding by section have a slight decrease as the percentage of people who didn't follow the rules increases. Boarding by section is the slowest way. The slight decrease indicates that people will have a higher chance putting their luggage simultaneously when they are in a random order than in order.

The results are interesting. An order by section is worse than no order (random). This shows that the boarding methods must try to decrease the factors of seat shuffle and luggage placing, rather than putting everybody in an organized order.

5.3 Situation of More Luggage

Many people might bring lots of luggage in order to save money from consignment. In this case, we have collected data at situation where two-thirds of people bringing luggage, four-fifths of people bringing luggage, and nine-tenths of people bringing luggage. The results are the following:



As more luggage is brought into the aircraft, the time it takes to board the aircraft gradually increases. People bringing their luggage is an act of saving money; however, this action also lowers the efficiency of boarding, and disembarking the aircraft.

5.4 Other Boarding Methods

Boarding Methods strengths and weakness can be summarized into 5 different elements:

1. Luggage Placing: When a person places their luggage the whole line must stop.
2. Seat Shuffle: When a person is blocking another person's seat, they must come out and swap in order to let seat all the people.
3. Family: Family need to be grouped together closely, so that children can be taken care.
4. Groups: Numerous Groups can make the boarding very complicated.
5. Rowdiness: Boarding in chaos may lower the efficiency.

1. Steffen Method (which we are not using):

Met: Luggage Placing, Rowdiness, Seat Shuffle.

The fastest way of boarding the plane is presented by Jason H. Steffen, called the Steffen method. This involves the line of people going into adjacent rows and putting their luggage simultaneously. Then, another line of people come in, putting their luggage simultaneously, etc.

This is the best way to avoid the problem of luggage placing, and seat shuffle. In fact, it is the fastest way theoretically. However, it cannot meet the requirement of Family Groups. This makes this method unpractical [6].

2. Front back WMA Method

Met: Groups, Rowdiness, some Luggage Placing, some Seat Shuffle

This Method involves first dividing the airplane into front and back, and then divide the sections into window, middle, aisle seats. According to the order, back W, front W, back M, front M, back A, front A, the passengers sit in.

The dividing of groups isn't complicated and very practical. It can also, in some way, it decreases the time needed to wait for luggage placing and seat shuffle.

According to the model, the average time it takes to board is 464 second.

3. Grouping Method

Met: Seat Shuffle, Groups (the groups can be changed), some Family,

This method involves dividing the aircraft into sections. In this case, it is a 2×6 . The people are sitting in this order (1 and 2 go in simultaneously, 3 and 4 also go in simultaneously, etc.):

4	1
8	5
11	9
10	12
6	7
2	3

The dividing of groups is more complicated. Also, the sections can change when something is needed. It met some of the family, but it didn't met rowdiness.

According to the model, the time it takes to board is 459 sec.

In all, our optimal boarding method is the Front Back WMA method. With the time of boarding the same, Front Back WMA have a better organized structure than grouping method.

5.5 Optimal Disembarking Method

Not much of the disembarking method are possible. We have chosen the fastest way, as disembarking the plane needs less a "family group".

Line by Line Method:

When the plane lands, one person sitting on the aisle seats (C or D) of each row will come out and take their luggage. The person that come out on the first row will leave first, then the other five people on the same row will leave. The second row is the same as the first, and the pattern will be the same row after row until the people from the last row depart the plane.

This method is the fastest method: it avoids the situation of getting the luggage and seat shuffle. According to the model, disembarking the aircraft takes around 144 second.

VI. Other Passenger Aircraft

6.1 Flying Wing Passenger Aircraft

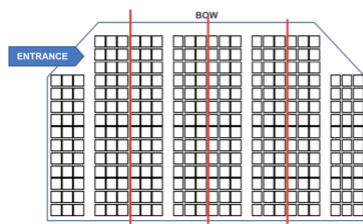


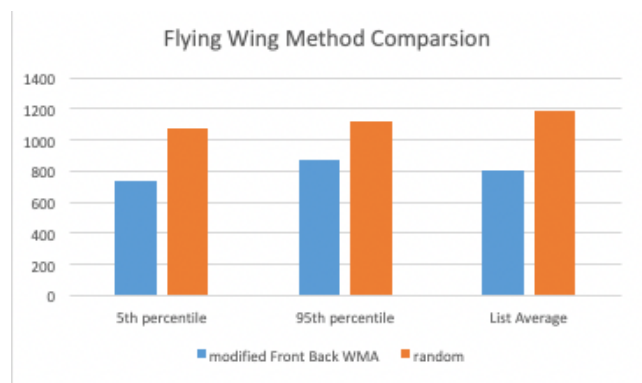
Figure 2. "Flying Wing" Passenger Aircraft

We have decided to divide the passenger aircraft into 4 narrow body passenger aircraft, and use the Front Back WMA method. The aircraft will be divided as below.

The aircraft will be divided into four "narrow body passenger aircraft. The furthest will be seated first, then one adjacent to it, then the one that is adjacent to it, then the one that is closest to the entrance.

Using a model to estimate the time, we get that the method will need around 801 second. Comparing with random, this method is 389 second quicker. More detailed information is listed to the right:

For the disembarking method, we will still use the line by line method. We divided the passenger aircraft into four narrow body passenger aircraft, and then use the line by line method on all of the "narrow body passenger aircraft".

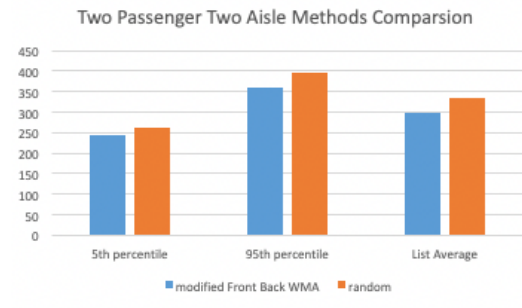


6.2 Two Entrance Two Aisle Passenger Aircraft

For the two entrance two aisle passenger aircraft, we have modified the Front Back WMA Method. First, we divide the whole airplane to the Royale class, front economic class, and back economic class. For the front economic class and back economic class, the seats will be further divided into aisle seats and "window seats" (the middle of the middle column of the seats counts as window seats). The Royale class enters first. Then, the back and front

economic class enters using the two entrances. In the order of window, aisle, the passengers' seat in.

Using a model to estimate the time, we get that the method will need around 300 second. Comparing with random, this method is 35 second quicker. More detailed information is listed on the table to the left.



For the disembarking method, we will also, still use the line by line method. In this case, the whole aircraft will be leaving through two entrances, so it will be divided into front and back. It also has two aisles, so the passengers will leave on both two aisles, where at the last time, there is only one aisle needed.

VII. Pandemic Situations

Decreasing the amount of people to 70%, 50%, or 30% will allow more of the space on the aircraft to be used. By this we made a method to use up the extra space. First, the even number of rows will be seated at window and aisle seats. If there are more people, then the odd number of rows will be seated at window and aisle seats. If there are still more people that haven't been seated, then, they will seat in the middle seats of each row. By this way the space can be most reasonably used, and also keep the most space between people and people.

This method is not putting time as the first priority. When the number of people decrease, the time also decrease rapidly. In this case, we more need to think of some needs: Is the space enough? Is the method more organized? In all, this space method is our recommended boarding method.

Our disembarking method stays the same. Line by Line Method is the best way to solve disembarking method. It is well organized, uncomplicated, and have maximum efficiency.

VIII. Letter

To the Airline Executive,

As the times change, fast transportation has become an important aspect needed to be achieved in order to ensure both efficiency and timeliness, and travelling by plane is undoubtedly the fastest way for movement for the people. Therefore, executive, our team has divided the process of being on a plane into three categories: boarding, the flying time, and disembarking periods, and since the flying time cannot be decreased since the legal cruising speed is between 550 and 580 mph [7], we layered out the different boarding and disembarking methods, and which after we observed and ran simulations taking specific details into account and comparing both the disadvantages and advantages of each, got the optimal boarding and disembarking methods for the standard narrow-body plane.

For boarding, the method which our team designated, the Front Back WMA method, is the best way, considering both practicality and efficiency. It divides the passengers into six groups: back window, front window, back middle, front middle, back aisle, and front aisle. The groups would enter the plane chronologically in the order as it was listed above. After simulation, our team found out that it was the method which took the least time apart from random (which we did not include due to the excess amount of rowdiness which would ensure shortly after boarding), and also had benefits like a small number of groups, making it easier to understand, and the evident lack of rowdiness, thereby maintaining the aircraft's order. By using this method, both efficiency and the practicality of it actually being followed by the majority is balanced out.

For disembarking, the best way was the Line by Line method, easy to understand and enforce. It is divided upon the rows, where the people in the front row would get out, first the aisle seats, then the middle seats, and afterwards the window seats. The process would be repeated for each row behind the empty ones, until the entire plane is evacuated. By doing so, the people's luggage would be easy to find and carry. There would not be people trying to go past the other, and families would stay together in a closer unit.

In conclusion, executive, please know that our team has worked hard and in detail, considering almost all of the details in which could delay the boarding and disembarking processes. The two methods, Front Back WMA, and the Line by Line method, are a combination of both timeliness and the realistic aspect of the method. Therefore, rest assured to apply those methods as the boarding and disembarking ways for the standard narrow body plane. Thank you!

Regards,
Team 22272397

IX. Strengths

- 9.1 Firstly, before our process in the actual formulation of the model, our team made lists of potential factors which would affect the total time needed for all the passengers to sit on the plane like the age, walking speed, strength, etc. Abundant research was done in order to make sure the statistics and data were all concise and accurate. A realistic model is strived to be achieved in order to provide the best boarding and disembarking method in real life; therefore, the correct numbers can ensure a flexible model which can be applied to many different scenarios.
- 9.2 Second, when our team was comparing the different types of both boarding and disembarking methods, we took into account various kinds of aspects which could cause delays for the passengers or clotting in the line, branching out into smaller areas in order to ensure that the methods would be a balance between the aspects and not just an evident inclination towards one, say, efficiency. By doing so, it is promised that our methods can be more inclusive and realistic.

X. Weaknesses

- 10.1 We assumed that the time for one passenger to lift one piece of luggage onto the overhead cabins to be five seconds. However, it is evident that in real life different people's strength vary, so by making the assumption the model may be reduced in the realistic aspect. However, albeit this assumption, the model would still be more accurate than if we simply "guessed" the time needed for each passenger to lift their luggage.
- 10.2 There is one other weakness which our team has, and yet even if it is impossible to achieve, it is still a flaw which decreases the realistic aspect of the method, and that is the factor of whether or not the passenger would stand up after they are seated and take additional things from their luggage placed on the overhead bins. There are endless possibilities to this situation, so therefore our team has deleted this possible scene in our simulation, but however it is not probable that there would be numerous people standing up after they have seated themselves, so therefore the results will not be affected greatly.

XI. Appendix

1. Flying Wing

```
import random, itertools

SECTION = 4
COLUMN = 6
ROW = 14
TIME_SWAP = 6.192 # Estimated through experiment

# destination = (section, column, row)
class passenger(object):
    def __init__(self, number, destination, age, luggage_weight=1):
        self.number = number
        self.destination = destination
        self.spd = spd_estimate(age)
        self.luggage_weight = luggage_weight * 5
        #luggage weight represents the time(s) to put the luggage into overhead bins

def time_to_destination(destination, spd):
    # distance / spd = time
    # length of pitch = 32in. = 0.8128m
    return (destination[2] + abs(3 - destination[1])) * 0.8128 / spd

def print_matrix(matrix):
    for i in range(len(matrix)):
        print(matrix[i])

def spd_estimate(age):
    # Expression see "Estimation of Speed with Age.py"
    return 6.056*10**(-6)*age**3 - 0.0009286*age**2 + 0.04178*age + 0.88

def not_sort(l, begin, end):
    for i in range(begin, end + 1):
        l.insert(random.randint(begin, end), l.pop(i))

def board_by_section_WMA():
    possible_destination = []
    temp = []
    for section in range(SECTION - 1, -1, -1):
        for col in (0, 5, 1, 4, 2, 3):
            for row in range(ROW):
                temp.append((section, col, row))
            if col in (5, 4, 3):
                not_sort(temp, 0, len(temp) - 1)
                possible_destination += temp
                temp = []
```

```
for row in range(3):
    for col in range(3):
        possible_destination.remove((0, col, row))
        possible_destination.remove((3, 5 - col, row))
    return possible_destination

def board_by_random():
    possible_destination = []
    for section in range(SECTION):
        for col in range(COLUMN):
            for row in range(ROW):
                possible_destination.append((section, col, row))
    not_sort(possible_destination, 0, len(possible_destination) - 1)
    for row in range(3):
        for col in range(3):
            possible_destination.remove((0, col, row))
            possible_destination.remove((3, 5 - col, row))
    return possible_destination

seating_chart = []
for i in range(4):
    section = []
    for c in range(COLUMN):
        section.append([0] * ROW)
    seating_chart.append(section)
average = []
for DATA in range(5):
    L = []
    for TIMES in range(100):
        pList = []
        tList = [0] * 318

        # Create seating order (318)
        temp = []
        possible_destination = board_by_section_WMA()
        disembark = {}
        for p in range(318):
            pas = passenger(p + 1, possible_destination[p], random.randint(10, 60),
random.randint(0, 1))
            pList.append(pas)
            t = time_to_destination(pList[p].destination, pList[p].spd)
            tList[p] += t
            if (pList[p].destination[0], pList[p].destination[1]) not in disembark:
                disembark[(pList[p].destination[0], pList[p].destination[1])] = t
            elif t > disembark[(pList[p].destination[0], pList[p].destination[1])]:
                disembark[(pList[p].destination[0], pList[p].destination[1])] = t
```



```

# Separate Calculations for Line-by-Line Disembark
d1 = max([disembark[(0, 2)], disembark[(0, 3)], disembark[(1, 2)], disembark[(1, 3)],
disembark[(2, 2)], disembark[(2, 3)], disembark[(3, 2)], disembark[(3, 3)]])
d2 = max([disembark[(0, 1)], disembark[(0, 4)], disembark[(1, 1)], disembark[(1, 4)],
disembark[(2, 1)], disembark[(2, 4)], disembark[(3, 1)], disembark[(3, 4)]])
d3 = max([disembark[(0, 0)], disembark[(0, 5)], disembark[(1, 0)], disembark[(1, 5)],
disembark[(2, 0)], disembark[(2, 5)], disembark[(3, 0)], disembark[(3, 5)]])
print('Disembark in:', d1 + d2 + d3 + 3 * 5)

seating_chart = []
for i in range(ROW):
    seating_chart.append([0] * COLUMN * SECTION)

t = 0
while True:
    pas = pList.pop(t)
    pas_row = pas.destination[2]
    pas_col = pas.destination[1]
    pas_sec = pas.destination[0]
    pList.insert(0, None)
    tList[t] += pas.luggage_weight
    for p in range(len(pList)):
        if pList[p] is None:
            continue
        p_row = pList[p].destination[2]
        if p_row >= pas_row:
            tList[p] += pas.luggage_weight
        elif p > (pas_row - p_row):
            tList[p] += pas.luggage_weight

    if pas_col not in (2, 3):
        for col in range(COLUMN):
            if seating_chart[pas_row][col + 6 * pas_sec] != 0 and (pas_col < col <= 2 or
                                                                    pas_col > col > 2):
                tList[t] += TIME_SWAP
                for p in range(len(pList)):
                    if pList[p] is None:
                        continue
                    p_row = pList[p].destination[2]
                    if p_row >= pas_row:
                        tList[p] += TIME_SWAP
                    elif p > (pas_row - p_row):
                        tList[p] += TIME_SWAP
                seating_chart[pas_row][pas_col + 6 * pas_sec] = pas.number
    # print_matrix(seating_chart)
    # print()
    if t == len(pList) - 1:
        L.append(max(tList))
        break

```

```
        else:
            t += 1
    L.sort()
    print(L[5], L[95])
    average.append(sum(L)/100)
    print(sum(average)/5)
```

2. Narrow Body

```
import random, itertools, math
```

```
COLUMN = 6
```

```
ROW = 33
```

```
TIME_SWAP = 6.192 # Estimated through experiment
```

```
# destination = (column, row)
```

```
class passenger(object):
```

```
    def __init__(self, number, destination, age, luggage_weight=1):
```

```
        self.number = number
```

```
        self.destination = destination
```

```
        self.spd = spd_estimate(age)
```

```
        self.luggage_weight = luggage_weight * 5
```

```
        # luggage weight represents the time(s) to put the luggage into overhead bins
```

```
def time_to_destination(destination, spd):
```

```
    #distance / spd = time
```

```
    #length of pitch = 32in. = 0.8128m
```

```
    return destination[1] * 0.8128 / spd
```

```
def print_matrix(matrix):
```

```
    for i in range(len(matrix)):
```

```
        print(matrix[i])
```

```
        if i == 2:
```

```
            print('-' * 30)
```

```
def spd_estimate(age):
```

```
    #Expression see "Estimation of Speed with Age.py"
```

```
    return 6.056*10**(-6)*age**3 - 0.0009286*age**2 + 0.04178*age + 0.88
```

```
def not_sort(l, begin, end):
```

```
    for i in range(begin, end + 1):
```

```
        l.insert(random.randint(begin, end), l.pop(i))
```

```
def shuffle_by_percentage(l, p):
```

```
    swap = []
```

```
    for i in range(len(l)):
```

```
        swap.append(i)
```

```
    change = round(len(l) * p)
```

```
for i in range(change//2):
    a = random.choice(swap)
    swap.remove(a)
    b = random.choice(swap)
    swap.remove(b)
    l[a], l[b] = l[b], l[a]

def board_by_section():
    possible_row = list(range(1, 33))
    possible_row.remove(16)
    possible_destination = []
    for i in itertools.product(possible_row, [0, 1, 2, 3, 4, 5]):
        possible_destination.append((i[1], i[0]))
    possible_destination.remove((3, 1))
    possible_destination.remove((4, 1))
    possible_destination.remove((5, 1))
    not_sort(possible_destination, 117, 182)
    not_sort(possible_destination, 57, 116)
    not_sort(possible_destination, 0, 56)
    return possible_destination[::-1]

def board_by_seats():
    w, m, a = [(5, 1)], [(4, 1)], [(3, 1)]
    for row in range(2, 33):
        if row == 16:
            continue
        w.append((5, row))
        w.append((0, row))
        m.append((4, row))
        m.append((1, row))
        a.append((3, row))
        a.append((2, row))
    not_sort(w, 0, len(w) - 1)
    not_sort(m, 0, len(m) - 1)
    not_sort(a, 0, len(a) - 1)
    return w + m + a

def board_by_random():
    possible_row = list(range(1, 33))
    possible_row.remove(16)
    possible_destination = []
    for i in itertools.product(possible_row, [0, 1, 2, 3, 4, 5]):
        possible_destination.append((i[1], i[0]))
    possible_destination.remove((0, 1))
    possible_destination.remove((1, 1))
    possible_destination.remove((2, 1))
    random.shuffle(possible_destination)
    return possible_destination
```

```
def board_by_groups():
    group_row = 2
    possible_destination = []
    cur_row = ROW - 1
    for groups in range(math.ceil(ROW / group_row)):
        for col in [0, 1, 2, 5, 4, 3]:
            for sub_row in range(group_row):
                possible_destination.append((col, cur_row - sub_row))
            cur_row -= group_row
    possible_destination.remove((0, 1))
    possible_destination.remove((1, 1))
    possible_destination.remove((2, 1))
    return possible_destination
```

```
def board_by_FrontBack_WMA():
    bw, bm, ba, fw, fm, fa = [], [], [], [], [], []
    #back
    for row in range(17, 33):
        bw.append((0, row))
        bw.append((5, row))
        bm.append((1, row))
        bm.append((4, row))
        ba.append((2, row))
        ba.append((3, row))
    not_sort(bw, 0, len(bw)-1)
    not_sort(bm, 0, len(bm)-1)
    not_sort(ba, 0, len(ba)-1)
```

```
#front
    for row in range(1, 16):
        fw.append((0, row))
        fw.append((5, row))
        fm.append((1, row))
        fm.append((4, row))
        fa.append((2, row))
        fa.append((3, row))
    not_sort(fa, 0, len(fa)-1)
    not_sort(fm, 0, len(fm)-1)
    not_sort(fw, 0, len(fw) - 1)
```

```
    fw.remove((0, 1))
    fm.remove((1, 1))
    fa.remove((2, 1))
```

```
    return bw + bm + ba + fw + fm + fa
```

```
def board_during_pandemic(percent):
    # Forbidden Seats
```

```
fseats = [(3, 0), (4, 0), (5, 0), (3, 1), (4, 1), (5, 1), (0, 16), (1, 16), (2, 16), (3, 16), (4, 16), (5, 16)]
```

```
possible_destination = []
total_ppl = int(percent * 183)
row = ROW - 1
column = (0, 5)
flag = True
while total_ppl > 0:
    # Boundary Check
    if row < 0 and flag:
        if column == (0, 5):
            row = ROW - 1
            column = (2, 3)
        elif column == (2, 3) and (0, ROW - 2) not in possible_destination:
            row = ROW - 2
            column = (0, 5)
        elif (0, ROW - 2) in possible_destination:
            flag = False
            row = ROW - 1
            column = (1, 4)

    # Select Seat
    for col in column:
        if total_ppl == 0:
            break
        possible_destination.append((col, row))
        total_ppl -= 1

    # Switch Row
    if flag:
        row -= 2
    else:
        row -= 1

    # Forbidden Seat Check
    for f in fseats:
        if f in possible_destination:
            possible_destination.remove(f)
            total_ppl += 1
    return possible_destination
```

```
average = []
```

```
for DATA in range(5):
    L = []
    for TIMES in range(100):
        # Modifying Situations
        tList = [0] * 183
```

```
pList = []

# 1. Random
"""
possible_destination = board_by_random()
"""

# 2. Boarding by Section
"""
possible_destination = board_by_section()
"""

# 3. Boarding by Seat
"""
possible_destination = board_by_seats()
"""

# 4. Boarding by Groups
"""
possible_destination = board_by_groups()
"""

# 5. Boarding by Front-Back WMI
"""
possible_destination = board_by_FrontBack_WMA()
"""

# 6. Boarding during Pandemic
possible_destination = board_during_pandemic(1)

# Regarding the Passengers who Do Not Follow Direction
"""
shuffle_by_percentage(possible_destination, 0.7)
"""

# Varying the number of carry-on bags
carry_bags = []
for i in range(183):
    carry_bags.append(random.choice([1, 0]))

# Create Passengers
for p in range(183):
    pas = passenger(p+1, possible_destination[p], random.randint(10, 60), carry_bags[p])
    pList.append(pas)

# Algorithm
disembark = {}
for p in range(183):
    t = time_to_destination(pList[p].destination, pList[p].spd)
```

```

tList[p] += t
if pList[p].destination[0] not in disembark:
    disembark[pList[p].destination[0]] = t
elif t > disembark[pList[p].destination[0]]:
    disembark[pList[p].destination[0]] = t

seating_chart = []
for i in range(COLUMN):
    seating_chart.append([0] * ROW)

t = 0

while True:
    pas = pList.pop(t)
    pList.insert(0, None)
    tList[t] += pas.luggage_weight
    for p in range(len(pList)):
        if pList[p] is None:
            continue
        if pList[p].destination[1] >= pas.destination[1]:
            tList[p] += pas.luggage_weight
        elif p > (pList[p].destination[1] - pList[p].destination[1]):
            tList[p] += pas.luggage_weight

    for i in range(COLUMN):
        if seating_chart[i][pas.destination[1]] != 0 and (pas.destination[0] < i <= 2 or
                                                         pas.destination[0] > i > 2):

            # Seat Shuffle
            tList[t] += TIME_SWAP
            for p in range(len(pList)):
                if pList[p] is None:
                    continue
                if pList[p].destination[1] >= pas.destination[1]:
                    tList[p] += TIME_SWAP
                elif p > (pList[p].destination[1] - pas.destination[1]):
                    tList[p] += TIME_SWAP
            #seating_chart[pas.destination[0]][pas.destination[1]] = pas.number
            #print_matrix(seating_chart)
            if t == len(pList) - 1:
                break
            else:
                t += 1
    L.append(max(tList))
    print("Disembark in: ", sum(disembark.values()) + 5 * 6)
    L.sort()
    print('5th Percentile: ', L[4], '95th Percentile', L[94])
    average.append(sum(L)/100)

print('Average:', sum(average)/5)

```

3. Two-Entrance Two Aisle

```
import random, itertools
```

```
COLUMN = 7
```

```
TIME_SWAP = 6.192 # Estimated through experiment
```

```
# destination = (section, column, row)
```

```
class passenger(object):
```

```
    def __init__(self, number, destination, age, luggage_weight=1):
```

```
        self.number = number
```

```
        self.destination = destination
```

```
        self.spd = spd_estimate(age)
```

```
        self.luggage_weight = luggage_weight * 5
```

```
        #luggage weight represents the time(s) to put the luggage into overhead bins
```

```
def time_to_destination(destination, spd):
```

```
    # distance / spd = time
```

```
    # length of pitch = 32in. = 0.8128m
```

```
    row = destination[1]
```

```
    if row < 3:
```

```
        return (3 - row) * 0.8128 / spd
```

```
    elif row < 17:
```

```
        return (row - 3) * 0.8128 / spd
```

```
    else:
```

```
        return (39 - row) * 0.8128 / spd
```

```
def offboard_time(destination, spd):
```

```
    row = destination[1]
```

```
    if row < 9:
```

```
        return row * 0.8128 / spd
```

```
    elif 9 <= row < 17:
```

```
        return (17 - row) * 0.8128 / spd
```

```
    elif row < 29:
```

```
        return (row - 17) * 0.8128 / spd
```

```
    else:
```

```
        return (39 - row) * 0.8128 / spd
```

```
def print_matrix(matrix):
```

```
    for i in range(len(matrix)):
```

```
        print(matrix[i])
```

```
        if i in (1, 4):
```

```
            print('-' * 30)
```

```
def not_sort(l, begin, end):
```

```
    for i in range(begin, end + 1):
```

```
        l.insert(random.randint(begin, end), l.pop(i))
```



```
def spd_estimate(age):  
    # Expression see "Estimation of Speed with Age.py"  
    return 6.056*10**(-6)*age**3 - 0.0009286*age**2 + 0.04178*age + 0.88
```

```
def max4(n1, n2, n3, n4):  
    return max(n1, max(n2, max(n3, n4)))
```

```
def board_by_FrontBack_BMA():  
    # First Class  
    fir_win = []  
    fir_ais = []  
    for row in range(3):  
        fir_win.append(((0, row)))  
        fir_win.append(((3, row)))  
        fir_win.append(((6, row)))  
        for col in range(1, 6):  
            if col == 3:  
                continue  
            fir_ais.append(((col, row)))  
    not_sort(fir_win, 0, len(fir_win) - 1)  
    not_sort(fir_ais, 0, len(fir_ais) - 1)  
  
    # Economic Class  
    eco_win = []  
    eco_ais = []  
    for row in range(4, 38):  
        if row in (16, 17, 18):  
            continue  
        eco_win.append(((0, row)))  
        eco_win.append(((3, row)))  
        eco_win.append(((6, row)))  
        eco_ais.append(((1, row)))  
        eco_ais.append(((2, row)))  
        eco_ais.append(((4, row)))  
        eco_ais.append(((5, row)))  
    not_sort(eco_win, 0, len(eco_win) - 1)  
    not_sort(eco_ais, 0, len(eco_ais) - 1)  
    possible_destination = fir_win + fir_ais + eco_win + eco_ais  
    return possible_destination
```

```
def board_by_random():  
    possible_destination = board_by_FrontBack_BMA()  
    random.shuffle(possible_destination)  
    return possible_destination
```

```
average = []  
for DATA in range(5):  
    L = []  
    for TIMES in range(100):
```

```

possible_destination = board_by_random()

# Create Passengers
pList = []
tList = [0] * 235
disembark = {}
for p in range(235):
    pas = passenger(p + 1, possible_destination[p], random.randint(10, 60),
random.randint(0, 1))
    pList.append(pas)
    t = time_to_destination(pList[p].destination, pList[p].spd)
    ot = offboard_time(pList[p].destination, pList[p].spd)
    tList[p] += t
    if pList[p].destination[0] not in disembark:
        disembark[pList[p].destination[0]] = ot
    elif ot > disembark[pList[p].destination[0]]:
        disembark[pList[p].destination[0]] = ot

seating_chart = []
for col in range(COLUMN):
    seating_chart.append([0] * 39)

t = 0
while True:
    pas = pList.pop(t)
    pas_row = pas.destination[1]
    pList.insert(0, None)
    tList[t] += pas.luggage_weight
    for p in range(len(pList)):
        if pList[p] is None:
            continue
        p_row = pList[p].destination[1]
        if 17 > p_row >= pas_row > 3:
            tList[p] += pas.luggage_weight
        elif 17 < p_row <= pas_row < 39:
            tList[p] += pas.luggage_weight
        elif p_row <= pas_row < 3:
            tList[p] += pas.luggage_weight

# Seat shuffle (which will be prevented in our method)
for i in range(2):
    if seating_chart[i][pas_row] != 0 and (pas.destination[0] < i <= 2 or
pas.destination[0] > i > 4):
        tList[t] += TIME_SWAP
        for p in range(len(pList)):
            if pList[p] is None:
                continue
            p_row = pList[p].destination[1]
            if 17 > p_row >= pas_row > 3:

```

```

        tList[p] += TIME_SWAP
    elif 17 < p_row <= pas_row < 39:
        tList[p] += TIME_SWAP
    elif p_row <= pas_row < 3:
        tList[p] += TIME_SWAP
    seating_chart[pas.destination[0]][pas.destination[1]] = pas.number
    #print_matrix(seating_chart)
    #print()
    if t == len(pList) - 1:
        L.append(max(tList))
        break
    else:
        t += 1
    d = (max4(disembark[0], disembark[1], disembark[2], disembark[3])
        + max4(disembark[3], disembark[4], disembark[5], disembark[6])) + 4 * 5
    #print(disembark)
    print("Disembark in: ", d)
    L.sort()
    print(L[5], L[95])
    average.append(sum(L)/100)
print(sum(average)/5)

```

4. Estimation of Speed with Age

```

import numpy as np
import pylab
from numpy import polyfit, poly1d

def fitting():
    x = np.linspace(20, 70, 6)
    y = np.array([1.393, 1.458, 1.462, 1.393, 1.359, 1.330]) #Statistics from Hartford Hospital
    #https://academic.oup.com/ageing/article-pdf/26/1/15/78378/26-1-15.pdf

    z1 = np.polyfit(x, y, 3)
    p1 = np.poly1d(z1)
    print(p1) # printing out the desired equation
    y_pred = p1(x)
    pylab.plot(x, y, '*', label='original values')
    pylab.plot(x, y_pred, 'r', label='fit values')
    pylab.title('Comfortable Walking Speed')
    pylab.xlabel('Year-Old')
    pylab.ylabel('Walking Speed (m/s)')
    pylab.legend(loc=3, borderaxespad=0, bbox_to_anchor=(0, 0))
    pylab.show()

if __name__ == '__main__':
    fitting()

```

XII. Works Cited

- [1] “EFFICIENT | Meaning & Definition for UK English | Lexico.com.” *Lexico Dictionaries | English*, www.lexico.com/definition/efficient. Accessed 11 Mar. 2022.
- [2] “Average Child Height | Onaverage.co.uk.” *Onaverage.co.uk*, 2019, www.onaverage.co.uk/body-averages/average-child-height.
- [3] Harvey, Author Leslie. “What to Consider before Flying with a Lap Child.” *Trips with Tykes*, 22 May 2013, tripswithtykes.com/should-you-buy-a-seat-for-your-child-under-age-2/. Accessed 10 Mar. 2022.
- [4] Christensen, Tricia. “Have Seats on Airplanes Gotten Smaller?” *Info Bloom*, 9 Feb. 2022, <https://www.infobloom.com/have-seats-on-airplanes-gotten-smaller.htm#:~:text=On%20coach%20or%20economy%20class%20planes%2C%20pitch%20tends,the%20smallest%20pitch%2C%20and%20international%20flights%20the%20largest>.
- [5] Bohannon, Richard M. “Comfortable and Maximum Walking Speed of Adults Aged 20-79 Years: Reference Values and Determinants.” *Academic.oup.com*, <https://academic.oup.com/ageing/article/26/1/15/35875>.
- [6] Cimler, R. & Olše ičo á, K. Analysis Simulation of aircraft disembarking methods, AWERProcedia Information Technology & Computer Science. [Online]. 2013, 3, pp 867-872. Available from: <http://www.world-education-center.org/index.php/P-ITCS>. Proceedings of 3rd World Conference on Information Technology (WCIT-2012), 14-16 November 2012, University of Barcelon, Barcelona, Spain.
- [7] Mecca, Jillian. “How Fast Does a Commercial Jet Airplane Fly.” *Getawaytips.azcentral.com*, 5 Oct. 2017, getawaytips.azcentral.com/how-fast-does-a-commercial-jet-airplane-fly-12431955.html#:~:text=The%20average%20cruising%20speed%20is%20between%200550%20and. Accessed 13 Mar. 2022.